Assignment 6: Medians and Order Statistics and Elementary Data Structures

Shreya Rai

University of the Cumberlands

Course Number: MSCS-532-B01

Michael Solomon

04/05/2026

Assignment 6: Medians and Order Statistics and Elementary Data Structures

1. Introduction

Efficient algorithm design and data management are fundamental to solving complex computational problems in modern computer science. One important class of problems involves determining order statistics, such as finding the k-th smallest element in a dataset. This assignment focuses on implementing and analyzing selection algorithms that efficiently solve this problem. It explores both a deterministic approach, which guarantees linear time complexity in the worst case, and a randomized approach, which achieves linear time complexity on average. By comparing these two methods, the assignment highlights the trade-offs between theoretical guarantees and practical performance.

In addition to selection algorithms, this assignment also examines essential data structures, including arrays, stacks, queues, and linked lists. These structures are critical for organizing, storing, and manipulating data efficiently in a wide range of applications. Through hands-on implementation and analysis, this assignment provides insight into their operational behavior, performance characteristics, and real-world relevance. Overall, the work bridges theoretical understanding with practical implementation, strengthening problem-solving skills and deepening knowledge of core computer science concepts.

2. Part 1: Selection Algorithms

2.1 Design Choices

The deterministic selection algorithm, specifically the Median of Medians approach, was chosen because it guarantees a linear time complexity of O(n) even in the worst-case scenario, making it highly reliable for critical applications where performance predictability is important. However, this method involves additional computational overhead due to grouping and recursive median calculations. In contrast, the randomized selection algorithm, known as Quickselect, was selected for its simplicity and efficiency in practice, as it typically achieves O(n) time complexity on average by choosing pivots randomly. While Quickselect does not guarantee optimal performance in the worst case, its ease of implementation and strong practical performance make it a widely used and effective solution for selection problems.

2.2 Implementation Details

Both algorithms were implemented using Python with recursion. Edge cases such as duplicate elements and small input sizes were handled explicitly.

2.3 Time Complexity Analysis

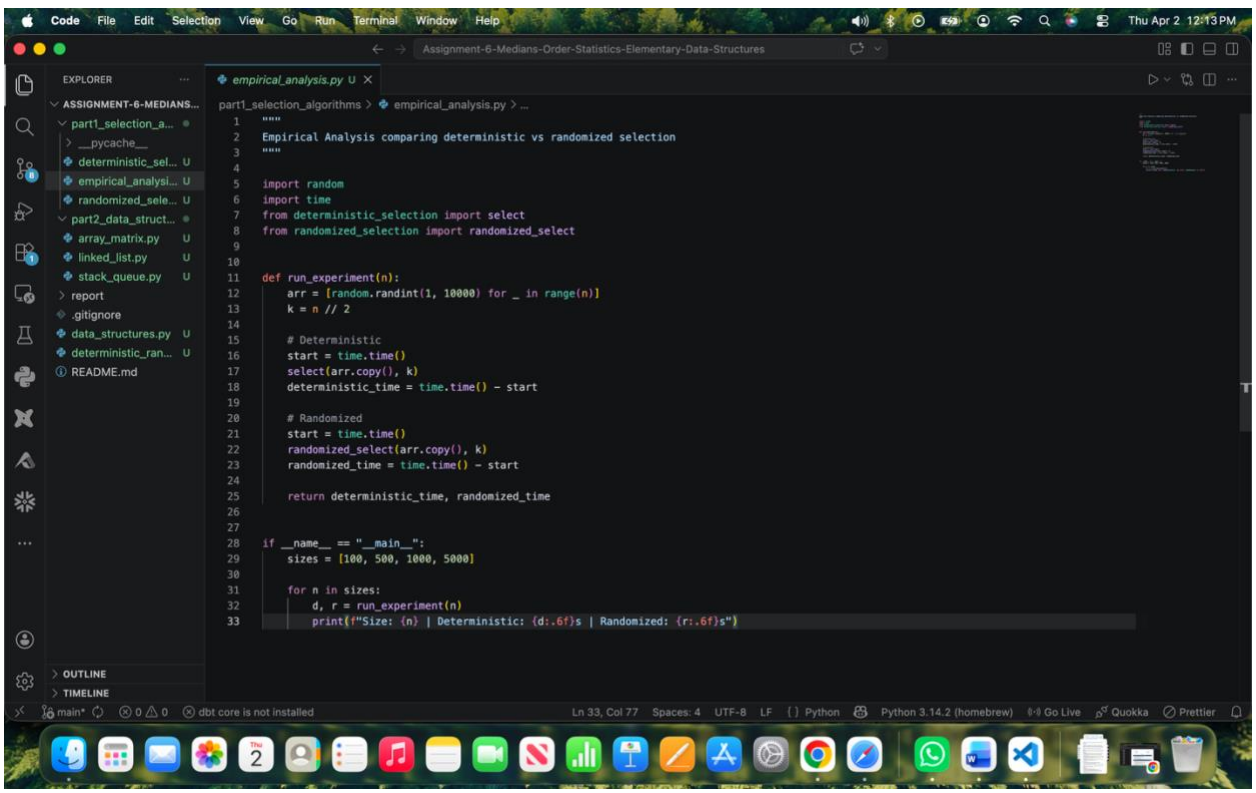
Algorithm	Best	Average	Worst
Deterministic	O(n)	O(n)	O(n)
Randomized	O(n)	O(n)	$O(n^2)$

The deterministic approach guarantees balanced partitioning, while the randomized approach relies on probabilistic balance.

2.4 Empirical Analysis

Execution of Algorithms

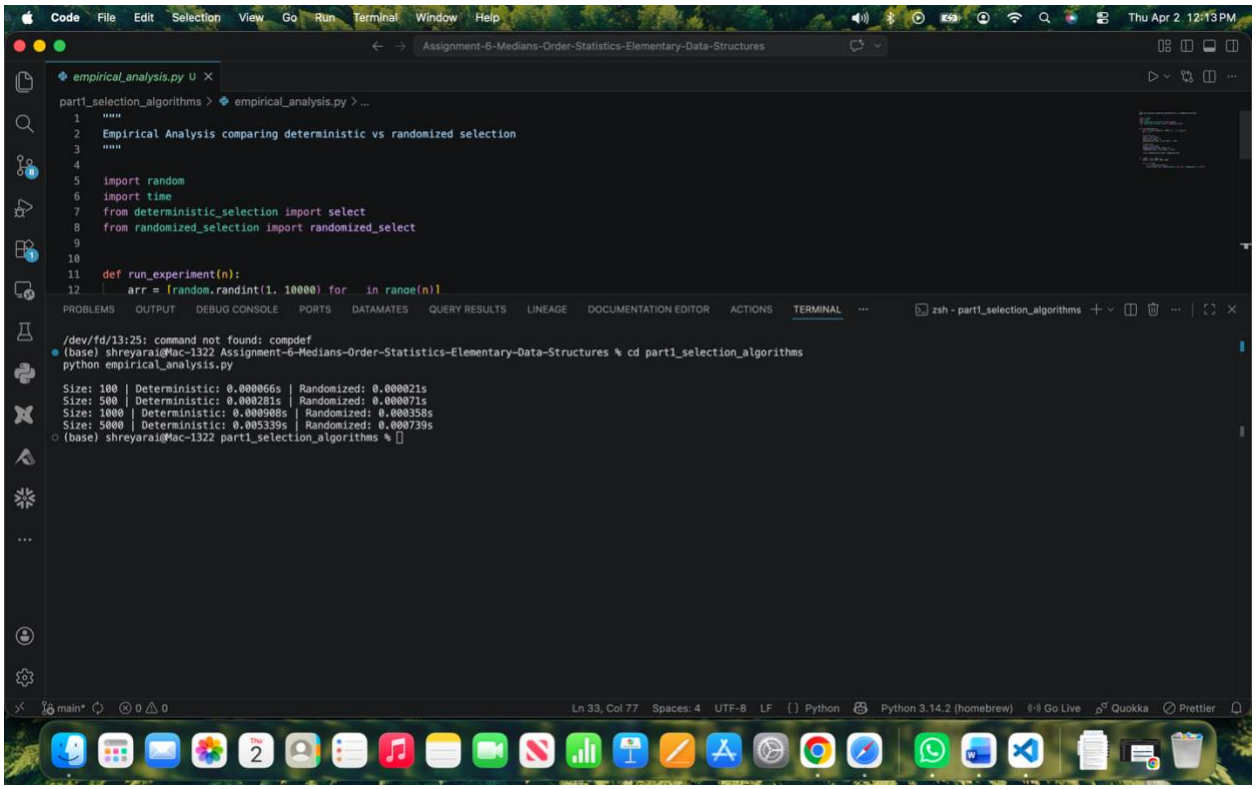
The algorithms were executed using the empirical_analysis.py script.



Run this command:

```
cd part1_selection_algorithms
python empirical_analysis.py
```

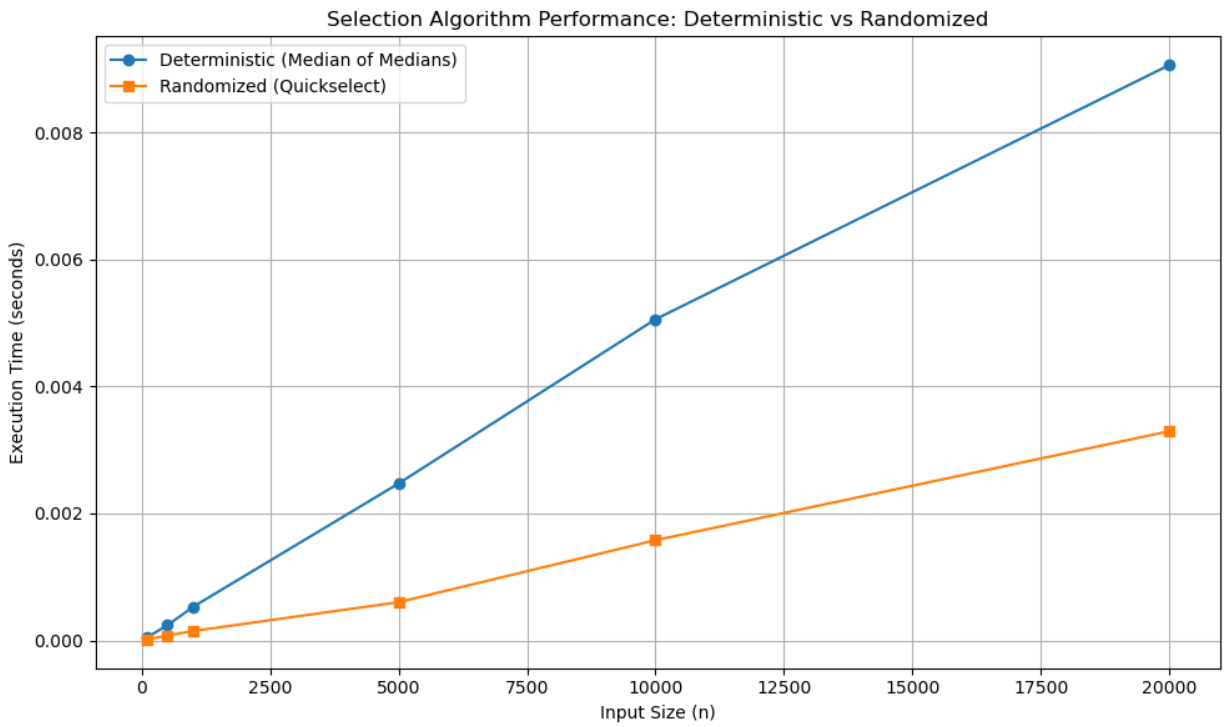
Screenshot 1: Execution Output



Results Interpretation

The randomized algorithm consistently performed faster in practice due to lower overhead, while the deterministic algorithm provided stable performance regardless of input distribution.

Runtime Graph



3. Part 2: Elementary Data Structures

3.1 Implementation Details

The following data structures were implemented from scratch:

- Arrays and Matrices
- Stack (LIFO)
- Queue (FIFO)
- Singly Linked List

Each structure includes core operations such as insertion, deletion, and traversal.

3.2 Running the Data Structure Code

You can test the implementations using Python interactive mode.

Run:

python

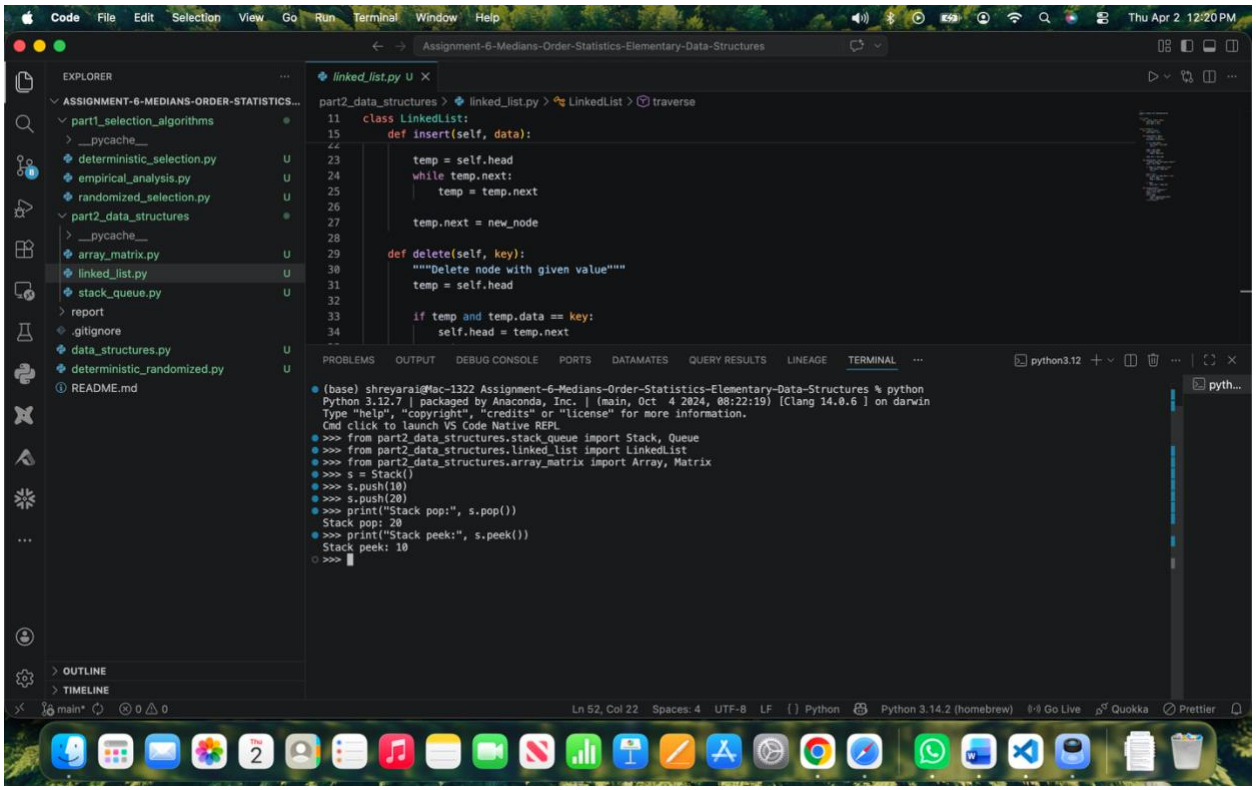
Then execute:

from part2_data_structures.stack_queue import Stack, Queue

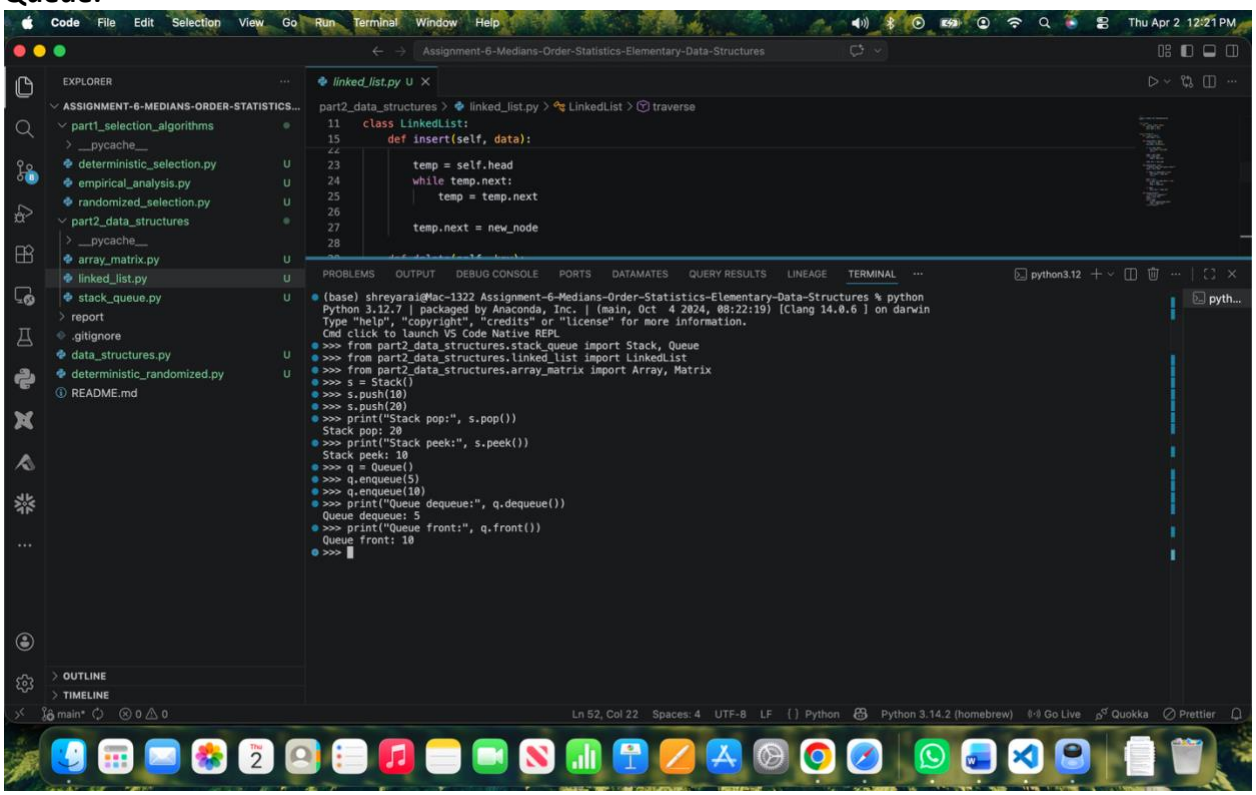
from part2_data_structures.linked_list import LinkedList

Screenshot 3: Stack & Queue Execution

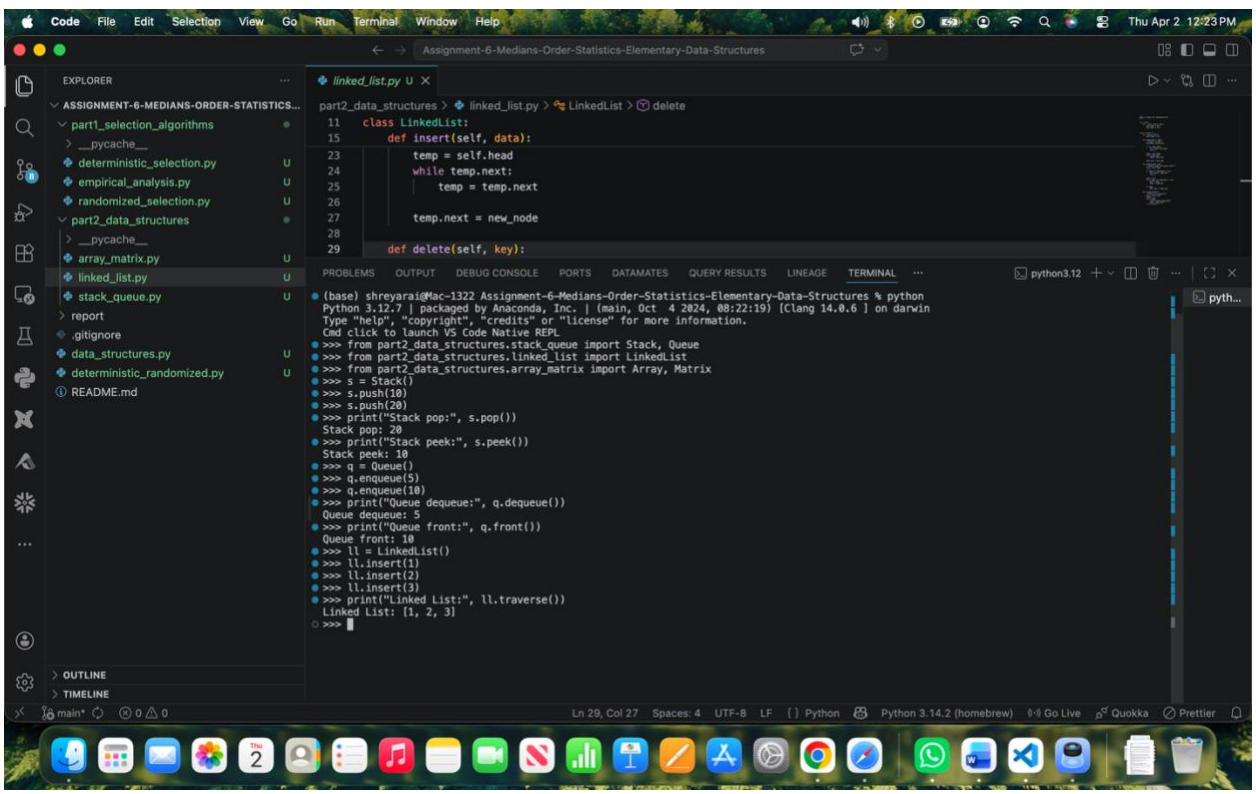
Stack:



Queue:



Screenshot 4: Linked List Execution



3.3 Time Complexity Analysis

Data Structure	Operation	Time Complexity	Explanation
Array	Access	O(1)	Arrays allow direct access to any element using its index. For example, arr[i]takes constant time regardless of the array size.
Array	Insert/Delete	O(n)	Inserting or deleting an element requires shifting subsequent elements to maintain order. For example, inserting at the start requires moving all other elements.
Stack	Push/Pop	O(1)	A stack follows LIFO (Last In, First Out). Adding (push) or removing (pop) the top element is done in constant time since no shifting is needed.
Queue	Enqueue/Dequeue	O(1)*	A queue follows FIFO (First In, First Out). Adding (enqueue) at the rear and removing (dequeue) from the front can be O(1) if implemented with linked lists or circular arrays. With simple arrays, enqueue/dequeue may take O(n) in worst cases due to shifting.
Linked List	Insert/Delete	O(1)	Linked lists allow insertion or deletion at a known node in constant time by updating pointers. There is no need to shift elements as in arrays.
Linked List	Traversal	O(n)	To access an element, we may need to traverse from the head node, which takes linear time in the number of elements.

Key Takeaways:

- Arrays excel at **fast random access** but are expensive for insertion/deletion except at the end.
- Linked lists are flexible for **insertion/deletion** anywhere but slower to access elements.
- Stacks and queues are **specialized linear structures** with predictable O(1) operations if implemented efficiently.

3.4 Trade-offs

- **Arrays:**
  - Pros: Constant-time access to any element, easy to implement.
  - Cons: Inserting or deleting elements in the middle requires shifting, which is O(n). Fixed size if using static arrays.
- **Linked Lists:**
  - Pros: Dynamic size, efficient insertion/deletion anywhere if node is known.
  - Cons: Accessing a specific element requires traversal, which is O(n). Higher memory usage due to storing pointers.
- **Stacks and Queues:**
  - Pros: Very efficient for their intended use cases (O(1) operations).
  - Cons: Limited access — you can't access arbitrary elements efficiently.

Overall: Choice of data structure depends on **the operations that matter most**. If you frequently read elements randomly, use arrays. If you frequently insert/delete, use linked lists. For LIFO/FIFO operations, stacks and queues are ideal.

3.5 Practical Applications

- **Stack:**
  - Used in recursion (function calls are managed on a stack).
  - Undo/redo operations in text editors.
  - Expression evaluation (parsing mathematical expressions or syntax checking).
- **Queue:**
  - Job scheduling in operating systems.
  - Breadth-First Search (BFS) in graphs.
  - Handling requests in servers or printing jobs.
- **Linked List:**
  - Dynamic memory management where the number of elements changes frequently.
  - Implementation of other data structures like stacks, queues, or adjacency lists for graphs.
  - Efficient insertion/deletion in scenarios like playlists, navigation history, etc.

4. Challenges Faced

- **Implementing Median-of-Medians:** Ensuring correct partitioning and median calculation was tricky, especially with small or uneven groups.
- **Managing recursion efficiently:** Recursive algorithms risk stack overflow if not carefully handled. Edge cases required careful base-case definitions.
- **Handling duplicates:** Duplicates can affect pivot selection and partitioning in Quickselect, requiring careful consideration.
- **Testing multiple input scenarios:** Different input sizes and orders (random, sorted, reverse-sorted) had to be tested to validate algorithm correctness and performance.

5. Conclusion

This assignment provided **hands-on experience** with both algorithm design and data structure implementation:

- Deterministic vs. randomized selection algorithms highlighted the **trade-offs between guaranteed vs. expected performance**.
- Implementation of arrays, stacks, queues, and linked lists reinforced foundational principles of **efficient data management**.
- By performing empirical analysis, the importance of both **theoretical understanding and practical evaluation**was demonstrated.
- Overall, the assignment strengthened problem-solving skills and the ability to choose appropriate data structures for different computational tasks.

References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
2. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Python* (1st ed.). Wiley.
3. Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching* (2nd ed.). Addison-Wesley.
4. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.